

Word embeddings

Limitations of the bag-of-words model

- many possible features
- high dimensionality
- ignores word order (sentence “*A is better than B*” is the same as “*B is better than A*”)
- one-hot-representation – no relations between words are captured

a ... guise guitar gulf ... plausible play player ... sober soccer sociability ... tenfold tennis tenon ... zucchini

0	...	0	1		...	0	1	0	...	0	0	0	...	0	0	0	...	0
0	...	0	0	0	...	0	1	0	...	0	0	0	...	0	1	0	...	0
0	...	0	0	0	...	0	1	0	...	0	1	0	...	0	0	0	...	0

play guitar

play tennis

play soccer

tens or hundreds of thousands
of dimensions

all vectors are equally
similar to each other

Eliminating some problems

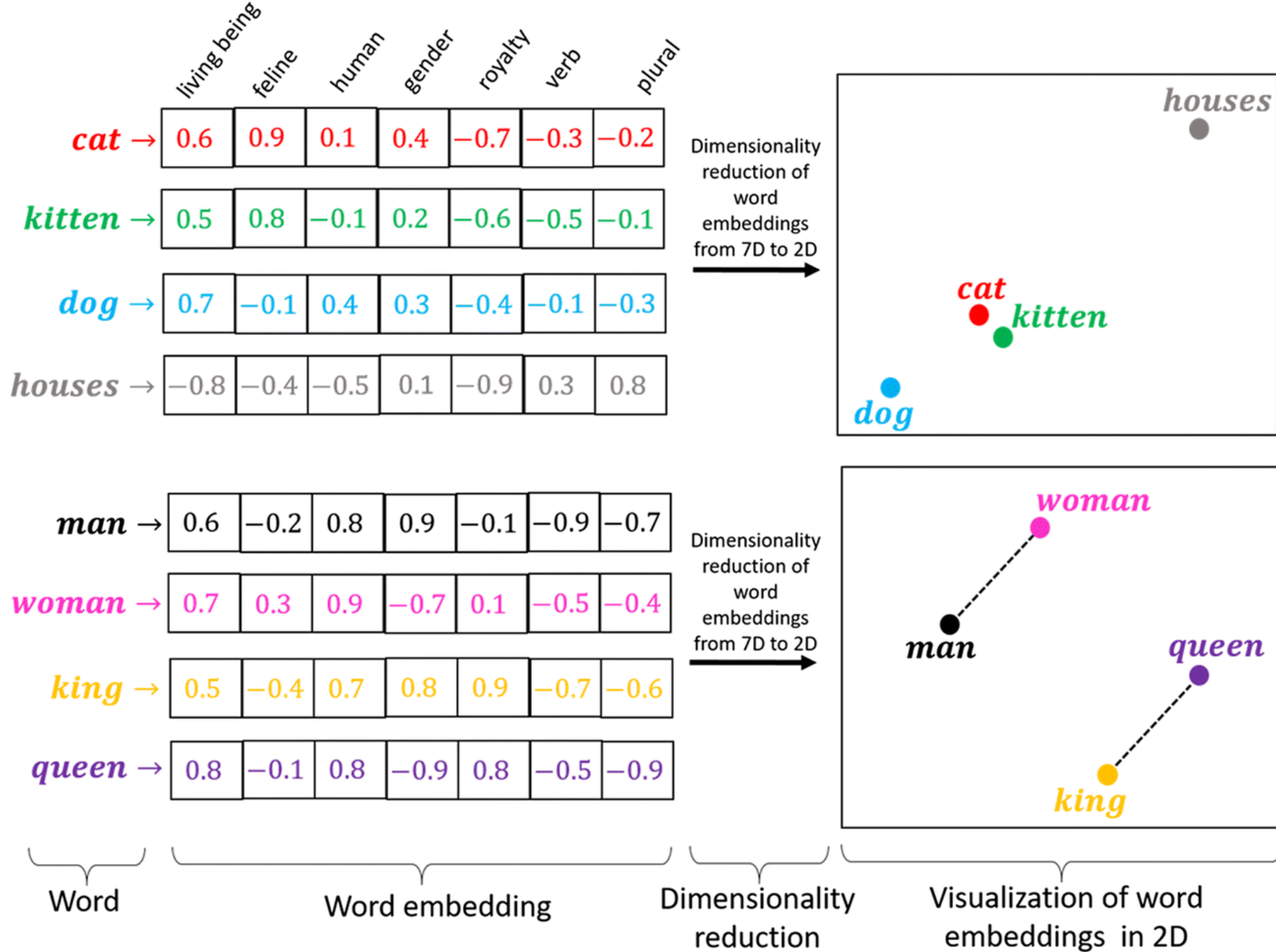
- feature selection
- dimensionality reduction (PCA, SVD)
- storing some positional information (context)
- new representations

Storing some context

- the word is A → the word is A, it's POS tag is T, the previous word is B, and the following word is C (leads to even more dimensions)
- representing a word by multiple dimensions and aggregating them

Word embeddings

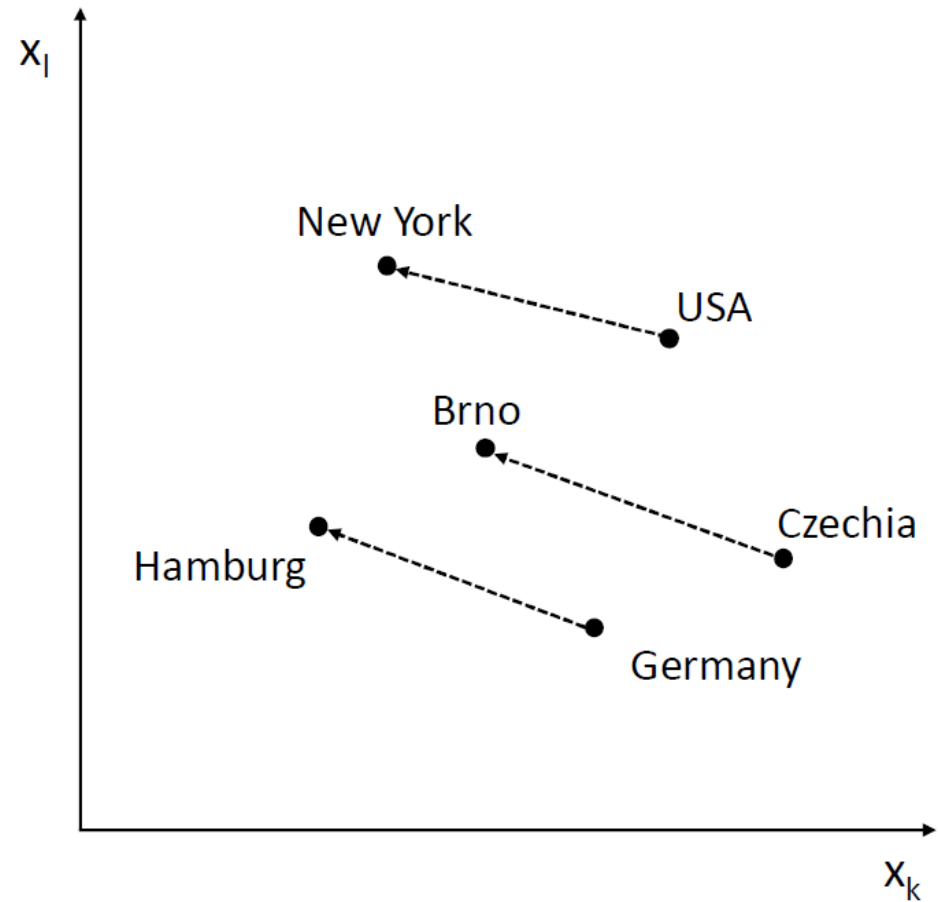
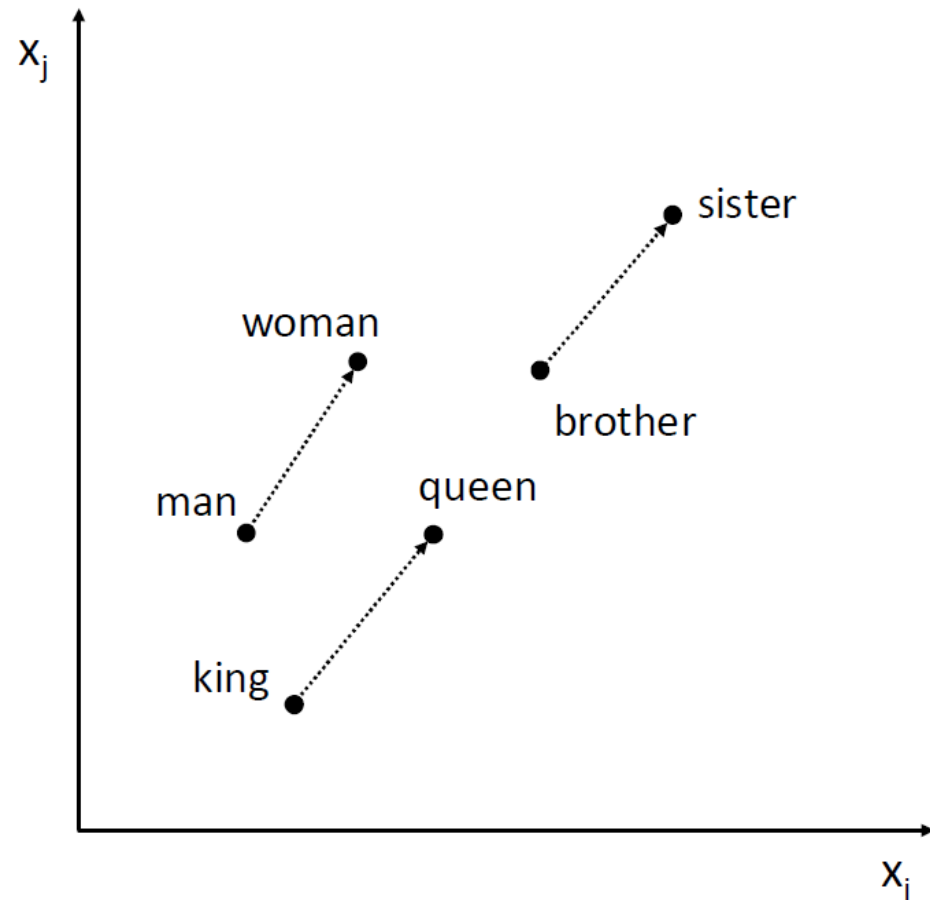
- based on the idea that similar words should have similar properties
- cannot be achieved in the bag-of-words model
- each word (or feature) = more dimensions
- words are mapped to (embedded in) a continuous multidimensional space (with typically a few hundred of dimensions) => we talk about *word embeddings*



Properties of word vectors

- similar words share some information => have similar vectors
- vectors operations can be performed with them, which can provide very surprising answers
- if there is a relationship between words A and B, and a similar relationship between words C and D, the offsets of A and B, and C and D are also similar

Properties of word vectors



$$\text{vector}(\text{"paris"}) - \text{vector}(\text{"france"}) + \text{vector}(\text{"poland"}) \approx \text{vector}(\text{"warsaw"})$$

Learning word embeddings

- it is difficult to manually define the word vectors' dimensions and assign values to even hundreds of thousands words → we want the process be automated
- the dimensions are difficult to interpret
- it must be defined what does it mean to be similar so the dimensions and values can be calculated
- words appearing in similar contexts usually have similar meaning (and thus are considered similar)

"A word is characterized by the company it keeps." (J.R. Firth, 1957)

The context

- a set of words in the environment of the word (the same document, sentence, or a piece of text of some length)
- context window
 - continuous bag-of-words (CBOW) – words in a window where no order or positions are considered (e.g., the context of X is A, B, C, and D)
 - positional – relative positions of other words are considered (e.g., the context of X is A two positions to the left, C one position to the left, D one position to the right, and C two positions to the right from X)
 - dependency – syntactic connections to the word are considered (e.g., A is the subject of X, B is an object of X)
- smaller windows (1-3 words) => more syntactic and functional similarities
- longer windows (4-10 words) => more semantic similarities

Computing word embeddings

- supervised methods
 - require annotated data for a specific task (e.g., texts with POS tags for the POS tagging task)
 - embeddings are trained toward the given goal and can capture information relevant to the task
 - embeddings can be used for a related task (e.g., syntactic parsing)
 - embeddings learned in one task can also be transferred to another task and improved with some labeled data from the other task

Computing word embeddings

- unsupervised methods
 - also called self-supervision – does not require annotated (labeled) data
 - their only goal is to compute embeddings
 - embeddings are usually learned in the task of predicting a word given its context or deciding, whether a word can belong to a context given examples of real and randomly created word-context pairs
 - large corpora can be used for training instead of annotated texts
 - embeddings capture general syntactic and semantic relationships and can be applied in a wide variety of tasks

Neural language model (Bengio et al., 2003)

- given a sequence of words $w_1 \dots w_T$, the goal is to learn a model $f(w_t, \dots, w_{t-n+1})$ that gives the probability $P(w_t | w_{t-n+1}^{t-1})$
- training = finding parameters of the function that optimize some error function

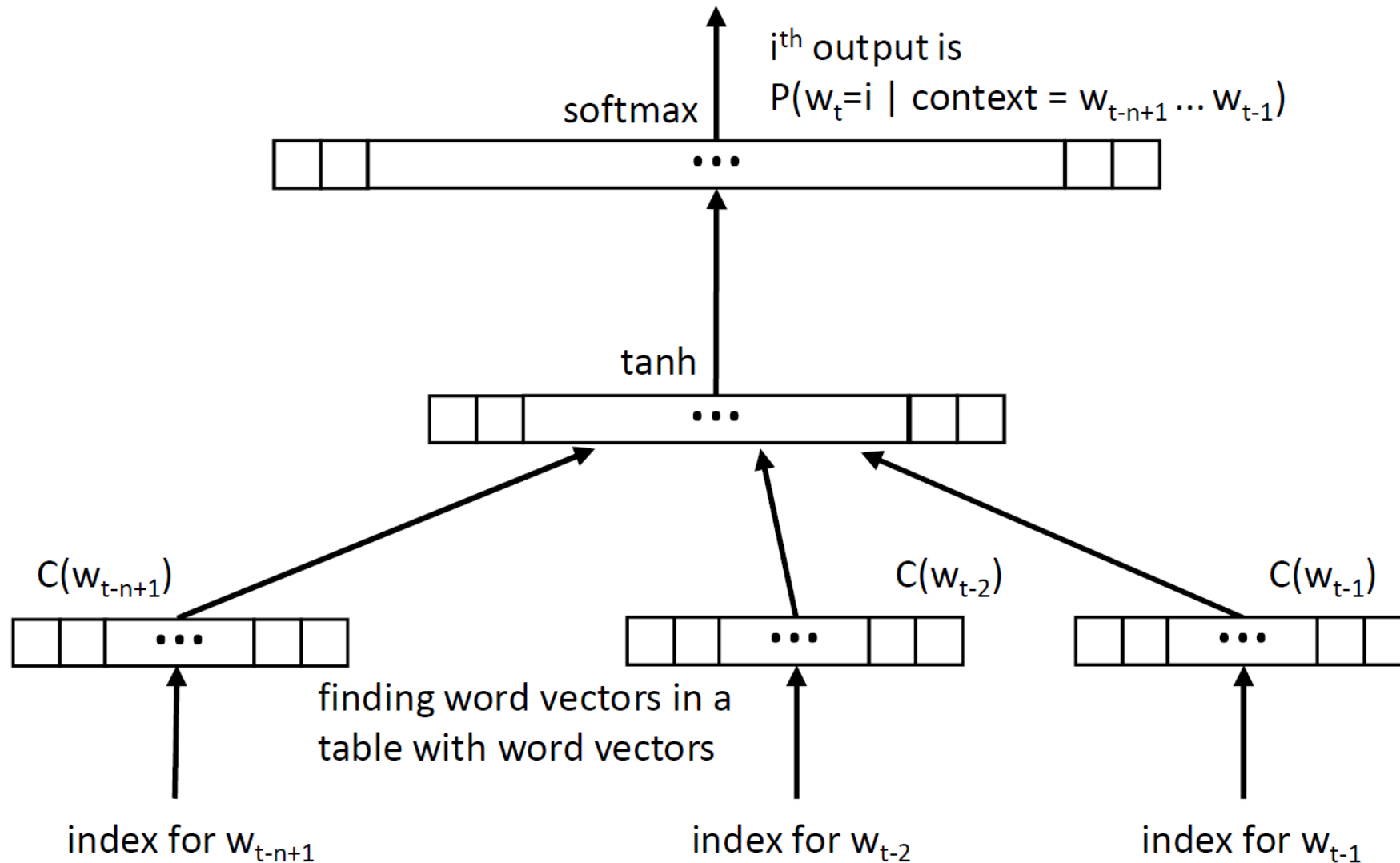
Neural language model (Bengio et al., 2003)

- mapping of i^{th} element from the vocabulary to a real vector $C(i)$, i.e., an embedding:
 - C ... embedding matrix with dimensions $d \times |V|$ (d ...embedding size, $|V|$...vocabulary size)
 - o ... one-hot vector of a word with dimensions $|V| \times 1$
 - $C \times o \rightarrow$ gives an embedding vector of a word with dimensions $d \times 1$
- the embedding vectors are concatenated and create an embedding layer with dimensions $Nd \times 1$ (N ...the size of context, d ... embedding size)
- hidden layer(s) follow (with weight matrices W_l)
- a softmax layer with dimensions $|V| \times 1$ at the end maps the input sequence $C(w_{t-n+1}), \dots, C(w_{t-1})$ to probability distribution over V

Neural language model (Bengio et al., 2003)

- during learning, typical phases are included – forward pass, cost function computation, backpropagation
- during the backpropagation phase, the matrix C is also updated

Neural language model (Bengio et al., 2003)



Word2vec

- a family of methods proposed by Tomáš Mikolov
- the prediction can have two forms
 - CBOW: predicting the word based on its context
 - Skip-gram: predicting the context for a word
- tries to eliminate the problems with the computational complexity of the previous neural language models:
 - a neural network uses linear computations (no tanh)
 - another loss is often computed (hierarchical softmax, negative sampling)

Word2vec

- CBOW
 - input embeddings are averaged into h
 - output embedding matrix is multiplied with h to get scores over the vocabulary (high number = the word is more likely the target)
 - transforming the scores using softmax to get probabilities

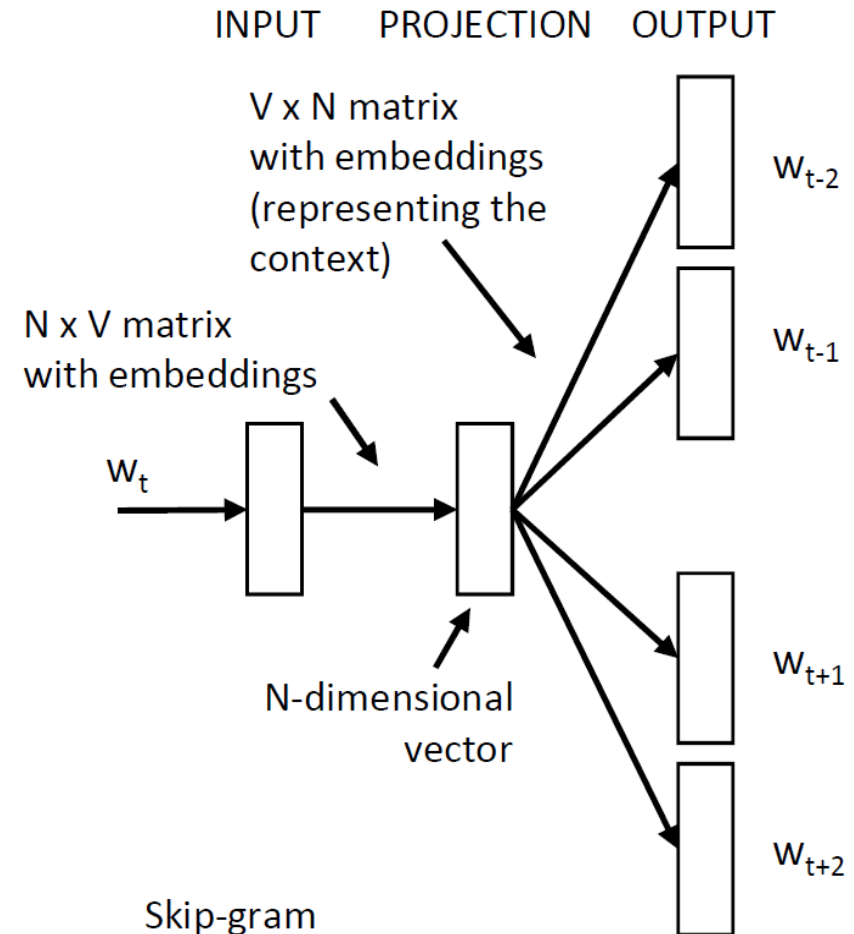
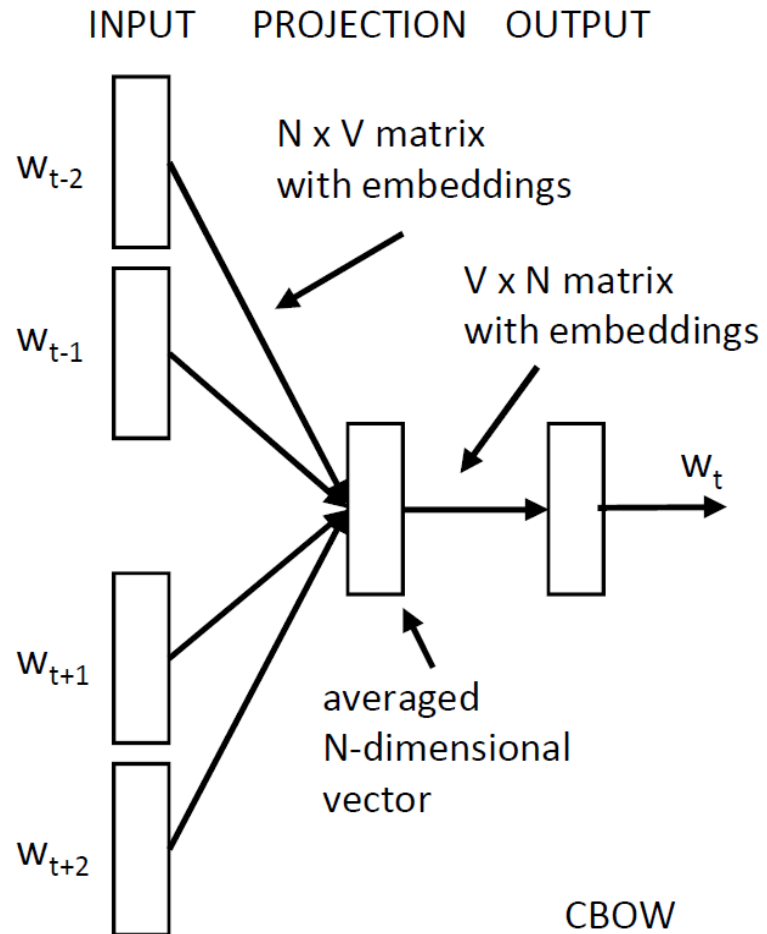
Word2vec

- skipgram
 - input embedding is multiplied with the output embedding matrix to get scores over the vocabulary (high number = the word is more likely the target) for each context word
 - transforming the scores using softmax to get probabilities
 - combining the scores in a cost function
 - we want to maximize the likelihood of context words (product of their probabilities) for each word in the corpus
 - minimizing the sum of logarithms of the probabilities is more common

Word2vec

- two embedding matrices are included in the computations
 - words play two roles (target/context)
 - this simplifies computations
 - both embeddings are averaged or only one is used

Word2vec



Word2vec

- instead of computing softmax over the entire vocabulary, negative sampling can be used
 - the model is given a correct example and a few (K , usually between 5 and 20) incorrect (negative samples)
 - negative samples are drawn randomly (not uniform distribution, common words have a higher probability than rare ones)
 - instead of softmax, another function is optimized:

$$Loss = -\log \sigma(v'_0 \cdot v_i) - \sum_{k=1}^K \log \sigma(-v'_k \cdot v_i)$$

- the score for the correct sample ($v'_0 \cdot v_i$) should be high and for incorrect samples ($v'_k \cdot v_i$) low

Word2vec

- skipgram
 - better handles rare words
 - in CBOW, the words are averaged, and the gradient update is thus distributed evenly
 - in skipgram, words are treated separately and each gets a full gradient update
 - enables having more training examples -> better learning
 - for one context window of size C , C training pairs are created
 - involves more computations than CBOW
 - often produces semantically richer embeddings

GloVe

- use the information about co-occurrences in the entire corpus (not only in smaller context windows)
 - X ... a co-occurrence matrix
 - X_{ij} ... how many times word i occurs in the context of word j
 - $X_i = \sum_k X_{ik}$... the number of times any word appears in the context of word i
 - $P_{ij} = P(j|i) = X_{ij}/X_i$... the probability that word j appears in the context of word i

GloVe

- single probability does not well capture the meaning or relations
- ratios using other words $P(k \mid i) / P(k \mid j)$ make a better job
 - ratio $\gg 1$... k is strongly associated with i
 - ratio $\ll 1$... k is strongly associated with j
 - ratio ≈ 1 ... k is not related to the semantic difference (does not help distinguish i and j)

GloVe

Probability and Ratio	$k = \textit{solid}$	$k = \textit{gas}$	$k = \textit{water}$	$k = \textit{fashion}$
$P(k \textit{ice})$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \textit{steam})$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \textit{ice})/P(k \textit{steam})$	8.9	8.5×10^{-2}	1.36	0.96

Co-occurrence probabilities for target words *ice* and *steam* with selected context words from a 6 billion token corpus. Only in the ratio does noise from non-discriminative words like *water* and *fashion* cancel out, so that large values (much greater than 1) correlate well with properties specific to *ice*, and small values (much less than 1) correlate well with properties specific of *steam*.

GloVe

- the ratio of probabilities should be computed using word vectors using some function F :
 - $F(w_i, w_j, w'_k) = P(k | i) / P(k | j)$
- after choosing a suitable F and the form of the relation between word vectors, the cost function can be derived:

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T w'_j + b_i + b'_j - \log X_{ij})^2$$

w ... embedding of a center word vector

w' ... embedding of a context word vector

b ... bias

f ... weighting function to reduce the importance of very frequent co-occurrences

GloVe

- the model is updated using the gradient descent method to find suitable parameters (word vectors)
- two sets of word vectors are produced (center, context), the final embedding is usually their sum

FastText

- most of the techniques for learning word embeddings ignore subword information (prefixes, suffixes), which is very important for morphologically rich languages with many rare words
- some forms of a word appear very rarely (or not at all) in a corpus
- word formations usually follow some rules – this can be exploited
- some previous attempts focused on deriving the representation of words using the morphemes (the smallest meaningful part of a word), which requires the knowledge of the morphology

FastText

- fastText (derived from word2vec) treats each word as a bag of subword units = character n-grams (one n-gram = one vector), e.g.,

where → <wh whe her ere re> <where>

- n-grams of different lengths are usually considered together, e.g.,

where → <wh whe her ere re> <whe wher here ere> <where>

- word vector = the sum of n-gram vectors
- in the task of predicting a center word (CBOW) or context (skipgram), often with negative sampling, subword embeddings are also learned
- enables creating vectors for words that are not in the training data

Word vector training

- vectors can be initialized randomly, like in the case of other model parameters
- values can be sampled uniformly from a fixed interval centered at zero

Using word vectors

- to represent larger pieces of text (sentences, paragraphs, documents), word vectors need to be aggregated so they have a fixed length
- simple aggregation approaches use, for example, sum, average, or a weighted average
- fixed length vectors can be an input to, for example, classification or clustering algorithms instead of document vectors from the bag-of-words model
- often represent words in complex ANN architectures (CNNs, RNNs, seq2seq)

Static embeddings

- the discussed embeddings are static
 - one word = one embedding
 - based on co-occurrence statistics
 - the same for words with multiple meanings (e.g., bank)
- contextual models (e.g., BERT, GPT) compute contextual embeddings
 - depend on other embeddings in the same context (e.g., sentence)
 - static embeddings serve as their input (initialized randomly or coming from word2vec or similar models) and can be updated in training

Using static embeddings

- instead of training, pre-trained embeddings can be found (trained on Wikipedia articles, tweets, Common Crawl, Google news and others)
- low memory and computational requirements, useful in low-resource environments
- useful for simpler NLP tasks where full contextual nuances are not critical
- can be used in baseline models
- using pretrained embeddings for the initialization of neural networks reduces training times